Rustlings 语法总结

> 进度快照：2026-10-20，已完成并验证 90/110 题，范围从 intro2 到 clippy3。编译题、测试题和 Clippy 题分别通过 48/48、39/39、3/3。
>
> 本文按语法主题整理，练习索引放在末尾。后续做完新题时，更新对应主题、索引和上面的进度数字即可；PDF 是此 Markdown 的阅读快照。尚未完成的 conversions、tests5 ~ 9 和 algorithm1 ~ 10 不计入本总结。

0. 知识地图

- 值与流程：变量、类型、函数、表达式、if、循环。
- 所有权与数据容器：移动、借用、Vec、String、HashMap。
- 组织代码：结构体、枚举、模式匹配、模块、泛型、trait、生命周期。
- 处理结果：Option、Result、自定义错误、测试、迭代器。
- 共享与并发：Box、Rc、Arc、Cow、线程、Mutex、通道。
- 元编程与质量检查：macro_rules!、Clippy。

做题时先问四件事：表达式是什么类型？谁拥有这个值？当前操作返回什么？调用方还能否继续使用原值？

1. 基础语法：变量、类型、函数与控制流

1.1 变量与类型

```
let x: i32 = 3;        // 不可重新赋值
let mut y = 3;         // 可修改绑定
y += 2;
let x = "three";       // 遮蔽：创建同名新绑定，类型可以变
const LIMIT: u32 = 40;  // 常量必须标注类型
```

变量默认不可变；`mut` 修改同一绑定，第二次 `let` 是遮蔽。`i32` 可表示负数，`u32` 不可；`usize` 常作长度和索引；`f32` 是浮点数。整数字面量可由上下文推断，也可写 `100u32` 明确类型。`as` 做显式数值转换，但不会替你检查业务条件，例如负数转无符号数之前仍需自行判断。

```
let flag: bool = true;
let ch: char = '中';          // 单引号：一个 Unicode 标量值
let array: [i32; 3] = [1, 2, 3];
let repeated = [0; 4];       // [0, 0, 0, 0]
let slice: &[i32] = &array[1..3];
let pair = ("Rust", 2026);
let year = pair.1;
```

数组 `[T; N]` 长度固定且属于类型的一部分；切片 `&[T]` 借用连续元素，长度可变。`[1..3]` 包含索引 1、2，不含 3。元组用 `.0`、`.1` 取字段，也可写 `let (name, year) = pair;` 解构。

1.2 函数、表达式和分支

```
fn square(n: i32) -> i32 {
    n * n                // 尾表达式：没有分号就是返回值
}

let label = if score >= 60 { "pass" } else { "fail" };
```

函数参数要标注类型；`->` 后写返回类型。末尾加分号会把表达式变成语句，函数得到单元值 `()`。`if` 是表达式，赋值时所有分支必须产生兼容类型。`==` 比较，`=` 赋值；`&&`、`||` 短路求值，`!` 取反，`%` 求余。

`0..3` 产生 0、1、2；`1..=3` 还包含 3。`for` 遍历可迭代值；`while let` 可在每轮循环重新尝试模式匹配。`panic!` 中断当前执行路径；可恢复的错误优先用 Result 表达。

1.3 格式化和常见宏

```
println!("x = {}", 5);      // Display
println!("items = {:?}", [1, 2, 3]); // Debug
let message = format!("x = {x}");   // 返回 String，不打印
assert_eq!(2 + 2, 4);
```

宏调用带 `!`。`println!` 打印，`format!` 构造字符串。自定义类型使用 `{:?}` 通常要派生 Debug，使用 `{}` 则要实现 Display。

2. 所有权、借用与 Vec

2.1 移动、复制、克隆

```
let a = String::from("hi");
let b = a;              // String 所有权移动，a 不能再用
let c = b.clone();      // 独立复制内容，b 仍可用
let n = 3;
let m = n;              // i32 实现 Copy，n 仍可用
```

按值传参也可能移动所有权；函数若返回该值，可把所有权交还调用者。`clone()` 会复制实际数据，不能把后来对副本的修改同步到原值。`drop(value)` 提前结束一个持有者的生命周期。

2.2 引用与可变性

```
fn length(v: &Vec<i32>) -> usize { v.len() }
fn add_one(v: &mut Vec<i32>) { v.push(1); }

let mut values = vec![1, 2];
let count = length(&values);
add_one(&mut values);
```

`&T` 是只读借用；`&mut T` 允许独占的可变借用。修改一个 Vec，既需要原绑定可变，也需要调用处传 `&mut`。同一时间不能同时持有冲突的可变引用与其他引用。`*reference` 解引用；方法和字段访问很多时候会自动解引用。`mut self` 表示方法取得整个值并在内部修改，`&mut self` 则只可变借用。

2.3 Vec 与切片

```
let mut v: Vec<i32> = vec![1, 2];
v.push(3);                // 原地修改，返回 ()
let last: Option<i32> = v.pop();
let len: usize = v.len();
let borrowed: &[i32] = &v[..];
for item in v.iter_mut() { *item += 1; }
```

`Vec<T>` 可变长，`pop()` 返回 Option，因为可能为空；`iter()` 产生 `&T`，`iter_mut()` 产生 `&mut T`。`sort()`、`clear()`、`resize()` 等原地方法返回 `()`；要先保存 Vec，再调用方法。`v.clone()` 会复制整个向量及其元素。

3. 字符串与映射

3.1 String、&str 与字符

`String` 拥有可增长的 UTF-8 文本；`&str` 只是借用文本。字符串字面量是 `&'static str`，但一般的 `&str` 不一定具有静态生命周期。

```
let mut s = String::from(" Rust ");
let trimmed: &str = s.trim();
let owned = trimmed.to_string();
s.push_str("lings");          // 原地修改
let upper = s.to_uppercase(); // 返回新 String
let replaced = s.replace("R", "r");
```

`trim()` 返回切片；`replace()`、`to_uppercase()` 返回新 String，不修改原值。`String + &str` 消耗左侧 String。不能用 `s[0]` 取得“首字符”，因为 UTF-8 字符占用的字节数不固定；可用 `chars()` 迭代 Unicode 标量值，`chars().last()` 返回 `Option<char>`。`lines()`、`split(',')` 返回切分后的迭代器；`"ab".repeat(3)` 返回新 String。

3.2 HashMap

```
use std::collections::HashMap;
let mut counts: HashMap<String, u32> = HashMap::new();
counts.insert("apple".to_string(), 2);
*counts.entry("apple".to_string()).or_insert(0) += 1;
let found: Option<&u32> = counts.get("apple");
let total: u32 = counts.values().sum();
```

`entry(key).or_insert(default)` 返回值的可变引用，解引用后可累计。`get` 返回 Option<&V>，不移走映射里的值。String 键可用 `&str` 查询，免去每次创建新 String。键常需 Hash + Eq；`#[derive(Hash, PartialEq, Eq)]` 可为合适的自定义类型派生这些能力。HashMap 遍历顺序不固定；需要确定顺序时先收集键再排序。

4. 结构体、枚举、匹配与模块

4.1 结构体和方法

```
struct Order { name: String, count: u32, urgent: bool }
struct Color(u8, u8, u8);   // 元组结构体
struct Marker;              // 类单元结构体

impl Order {
    fn new(name: String) -> Self {
        Self { name, count: 1, urgent: false }
    }
    fn count(&self) -> u32 { self.count }
}
```

`Self` 指当前实现的类型；`self` 消耗值，`&self` 只读借用，`&mut self` 可变借用。结构体更新语法 `Order { name: new_name, ..template }` 从模板取得剩余字段；非 Copy 字段可能因此移动。

4.2 枚举与模式

```
enum Message {
    Quit,
    Move(i32, i32),
    Write { text: String },
}

match message {
    Message::Quit => {},
    Message::Move(x, y) => println!("{x}, {y}"),
    Message::Write { text } => println!("{text}"),
}
```

枚举变体可不带数据、像元组、或带命名字段。`match` 必须穷尽所有可能分支；`_` 忽略内容，`x if x > 0` 是匹配守卫。`if let Some(x) = value` 适合只处理一种情况；`while let Some(x) = items.pop()` 可反复取值。`matches!(value, Pattern)` 返回布尔值。匹配拥有 String 的值时可能发生移动；后面还要使用原值时可考虑 `match &value`。

4.3 模块、路径与可见性

```
mod helper {
    pub fn run() {}
}
use helper::run;
run();
```

模块项目默认私有，跨模块使用时需 `pub` 并写对路径。`self::` 从当前模块起找，`super::` 从父模块起找，`crate::` 从 crate 根起找。`use std::time::{Duration, Instant};` 可分组导入；`as` 可重命名；`pub use` 可重新导出。

5. Option、Result 与错误传播

5.1 Option：可能没有值

```
let maybe: Option<i32> = Some(12);
if let Some(n) = maybe { println!("{n}"); }
let nothing: Option<i32> = None;
```

`Some(0)` 与 `None` 不同。`unwrap()` 遇 None 会 panic；能正常缺失的数据应先匹配或使用相应组合方法。若 `Vec<Option<T>>` 调用 `pop()`，结果是两层 `Option<Option<T>>`：外层表示 Vec 是否有元素，内层表示元素是否装着 T。

5.2 Result、? 与自定义错误

```
fn read_number(text: &str) -> Result<i32, std::num::ParseIntError> {
    let n = text.parse::<i32>()?;
    Ok(n)
}
```

`Result<T, E>` 用 Ok 表示成功、Err 表示失败。`?` 在 Err 时提前返回并按需要转换错误；在 Ok 时取出值，因此外层函数返回类型须兼容。`main() -> Result<(), E>` 可在 main 中用 `?`，最后返回 `Ok(())`。`map_err` 将底层错误包装为业务错误；自定义错误枚举可区分解析失败、规则不满足等情况。

`Result::unwrap()` 在 Err 时 panic，`unwrap_err()` 在 Ok 时 panic；测试中可用于断言预期，处理外部输入时通常应显式处理。要把不同具体错误作为同一种返回类型，可使用 `Box<dyn std::error::Error>`；自定义错误需实现 Display 和 Error。`fmt(&self, f: &mut fmt::Formatter) -> fmt::Result` 决定 Display 文本。

6. 泛型、Trait 与生命周期

6.1 泛型与约束

```
struct Wrapper<T> { value: T }
impl<T> Wrapper<T> {
    fn new(value: T) -> Self { Self { value } }
}

fn show(item: impl std::fmt::Display) -> String {
    format!("{item}")
}
```

泛型让一个定义适用多种具体类型。调用某个方法或用 `{}` 格式化时，需要给泛型加对应 trait bound；例如 `T: Display`。`impl TraitA + TraitB` 要求同一个实参同时实现两个 trait；具名写法是 `fn f<T: TraitA + TraitB>(x: T)`。约束多时可放到 `where` 后。两个参数若必须是同一具体类型，应共用一个 T。

6.2 定义与实现 Trait

```
trait AppendBar {
    fn append_bar(self) -> Self;
    fn label(&self) -> &str { "bar" } // 默认实现
}

impl AppendBar for String {
    fn append_bar(mut self) -> Self {
        self.push_str("bar");
        self
    }
}
```

无默认函数体的方法必须实现；有默认函数体时可以只写空实现块。方法签名里的 `self`、`&self`、`&mut self` 决定所有权。`#[derive(Debug, PartialEq)]` 只自动生成支持派生的 trait；自定义 trait 仍需手写 impl。Display 对应 `{}`，Debug 对应 `{:?}`。

6.3 生命周期

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

struct Book<'a> { title: &'a str }
```

生命周期标注描述引用之间的有效性关系，不会延长实际数据的存活时间。返回引用可能来自 x 或 y，使用它时两份实参都须仍有效。结构体保存引用时也需要标明生命周期；若要结构体独立拥有文本，则用 String 字段。

7. 闭包与迭代器

7.1 三种常见遍历方式

`iter()` 借用元素，得到 `&T`；`iter_mut()` 可变借用，得到 `&mut T`；`into_iter()` 消耗容器并移出元素。迭代器的 `next()` 每次推进一项，返回 Option，所以迭代器变量通常要 mut。

```
let doubled: Vec<i32> = [1, 2, 3].iter().map(|n| n * 2).collect();
let count = [1, 2, 3].iter().filter(|&&n| n > 1).count();
let factorial: u32 = (1..=5).product();
```

`map`、`filter`、`chain`、`flat_map` 建立惰性变换；`collect`、`count`、`sum`、`product` 消费迭代器。`flat_map` 把“每项产生的迭代器”展平成一条序列。`filter` 的闭包会再次借用当前项，因此 `values()` 给出 `&T` 时，闭包参数可能是 `&&T`；`|&&n|` 解开两层引用。

7.2 collect 的目标类型

```
let values: Vec<i32> = (1..=3).collect();
let parsed: Result<Vec<i32>, _> =
    ["1", "2"].into_iter().map(str::parse::<i32>).collect();
```

`collect()` 要靠目标类型决定收集方式。`Result<Vec<T>, E>` 遇到首个 Err 就失败；`Vec<Result<T, E>>` 保留每项的 Ok 或 Err。迭代器用过一次就被消费，第二种结果需重新创建迭代器。`1..=0` 是空范围，`product()` 对空迭代器返回乘法单位元 1，对应 0!。

8. 测试语法

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn works() {
        assert!(2 < 3);
        assert_eq!(2 + 2, 4);
        assert_ne!(2, 4);
    }

    #[test]
    #[should_panic]
    fn rejects_invalid_input() {
        panic!("invalid");
    }
}
```

`#[cfg(test)]` 只在测试配置下编译，`#[test]` 标记测试函数，`use super::*` 导入父模块名称。`assert!` 检查布尔条件；`assert_eq!`、`assert_ne!` 比较值；`#[should_panic]` 检查函数确实 panic。测试应调用实际被测逻辑，而不是仅比较两个常量。Rustlings 的 test 模式会编译并执行测试。

9. 智能指针：Box、Rc、Arc、Cow

9.1 Box：固定大小的递归边界

```
enum List {
    Cons(i32, Box<List>),
    Nil,
}
let one = List::Cons(1, Box::new(List::Nil));
```

直接在 List 内嵌 List 会让编译器无法确定类型大小；`Box<List>` 是固定大小的指针，实际节点放在堆上。`Nil` 是空链表，`Cons(value, next)` 是非空节点。

9.2 Rc 与 Arc：共享所有权

```
let sun = std::rc::Rc::new(Sun {});
let another_owner = std::rc::Rc::clone(&sun);
let owners = std::rc::Rc::strong_count(&sun);
drop(another_owner);
```

`Rc::clone` 增加对同一对象的持有者，不复制 Sun；重新 `Rc::new(Sun {})` 则创建另一对象。`Rc` 适合单线程共享；跨线程时用 `Arc`，例如先 `Arc::new(numbers)`，再为每个线程 `Arc::clone(&shared)`。Arc 的引用计数是原子的，但不保证其内部数据的任意修改也是安全的。

9.3 Cow：需要写入时才复制

`Cow<'a, [i32]>` 可是 `Borrowed(&[i32])` 或 `Owned(Vec<i32>)`。从 `&slice[..]` 构造时先借用；从 Vec 构造时一开始就拥有。`to_mut()` 需要可变访问：若当前 Borrowed，先复制再变成 Owned；若当前 Owned，直接修改原有 Vec。

在 cow1 中，借用的正数切片从未调用 `to_mut()`，仍为 Borrowed；借用且有负数时变成 Owned；从 Vec 构造的两种情况始终是 Owned。“没有复制”不等于 Borrowed。

10. 线程：等待、共享状态、消息传递

10.1 spawn、move 和 join（threads1）

```
let handle = std::thread::spawn(move || 42u128);
let result: u128 = handle.join().unwrap();
```

`spawn` 立即返回 `JoinHandle<T>`；闭包尾表达式决定 T。`move` 使闭包按值捕获所需变量，整型等 Copy 值会被复制；它不自动深拷贝 Vec。`join()` 消耗句柄，等待子线程结束并返回 `thread::Result<T>`；`unwrap()` 取正常结果，线程 panic 时也会 panic。先启动所有线程再逐个 join，才能让工作并发推进；按句柄顺序收集结果不代表完成顺序相同。

10.2 Arc<Mutex<T>>（threads2）

```
let shared = std::sync::Arc::new(std::sync::Mutex::new(
    JobStatus { jobs_completed: 0 }
));
let child = std::sync::Arc::clone(&shared);
let mut guard = child.lock().unwrap();
guard.jobs_completed += 1;
```

Arc 让线程持有同一份状态；Mutex 同一时刻只允许一个线程修改。`lock()` 返回 MutexGuard，守卫离开作用域便释放锁。睡眠或耗时操作应放在加锁之前或释放锁之后。读共享字段也要加锁。各线程可能先后交错，所以中途打印的完成数未必按 1、2、3 递增；全部 join 后才能保证最终计数已稳定。

10.3 mpsc 通道（threads3）

```
let (tx, rx) = std::sync::mpsc::channel::<u32>();
let tx1 = tx.clone();
std::thread::spawn(move || tx1.send(1).unwrap());
std::thread::spawn(move || tx.send(2).unwrap());
for value in rx { println!("{value}"); }
```

`mpsc` 是“多个发送者、一个接收者”。第一个 move 闭包取得 tx 后，原变量不能再交给第二个线程，所以要先克隆发送端。`for value in rx` 等待消息，直到所有发送端都释放后结束；主线程若额外留着一个发送端，循环可能一直等待。threads3 的 Arc<Queue> 只负责两个发送线程读取同一队列；通道负责把数字送回主线程，收到的先后顺序不固定。

11. 声明式宏：macros1 ~ 4

```
macro_rules! my_macro {
    () => {
        println!("empty arm");
    };
    ($value:expr) => {
        println!("value = {}", $value);
    };
}

my_macro!();
my_macro!(7 + 1);
```

- 宏调用必须写 `!`，例如 `my_macro!()`，不能写成普通函数调用。
- `macro_rules!` 的每个“匹配臂”由模式与展开代码组成；`($value:expr)` 捕获一个表达式。相邻匹配臂用分号分隔。

- 此组练习中的局部声明式宏有词法作用域，通常要先定义再调用。宏放在子模块里时，题目用 `#[macro_use] mod macros { ... }` 将其带入父作用域；这与普通函数的 `pub` 可见性规则不同。
- `#[rustfmt::skip]` 表示让 rustfmt 跳过该项格式化，不影响宏匹配本身。

12. Clippy：警告背后的语义

Clippy 在编译检查之外指出常见错误和不清晰写法。本仓库的 clippy 题以 `-D warnings -D clippy::float_cmp` 运行，因此警告会使题目失败。

- clippy1：近似常量。`3.14f32` 是 PI 的低精度近似；用 `f32::consts::PI`。
- clippy2：遍历 Option。只在 Some 时做一次操作，用 `if let Some(x) = option`，表达意图比 `for x in option` 清晰。
- clippy3：确定会 panic 的 unwrap。对 None 调用 `unwrap()` 一定 panic；用模式匹配处理 Some/None。
- clippy3：数组分隔符。数组元素之间必须有逗号；缺逗号的 `-3 -4` 会被解析为减法表达式。
- clippy3：原地方法的返回值。`Vec::resize()` 和 `clear()` 返回 `()`，不能用它们的返回值当 Vec；先创建可变 Vec，再调用 `clear()`。
- clippy3：交换变量。连续写 `a = b; b = a;` 会丢失 a 的旧值；用 `std::mem::swap(&mut a, &mut b)`。

13. 做题时的排查顺序

1. 定位类型：实际得到的是 T、&T、Option<T>、Result<T,E>、还是 ()？重点看方法的返回类型。
2. 检查所有权：传参、闭包捕获和赋值是否移动值？需要借用、共享指针，还是独立 clone？
3. 检查返回表达式：函数末尾有没有多余分号？if / match 各分支类型是否一致？
4. 检查可变性：是否有 `mut`、`&mut` 或 MutexGuard？调用的方法是原地修改还是返回新值？
5. 检查能力与作用域：泛型是否具备所需 trait？跨模块路径与可见性是否正确？宏是否已经定义并导入？
6. 验证行为：先按题目模式编译或运行测试，再看 Clippy；线程输出顺序可能变化，应验证最终结果。

14. 已完成练习索引（90/110）

下面的名单按 `info.toml` 顺序列出，均在 2026-10-20 通过相应的编译、测试或 Clippy 检查。每组名称后括号内是题数。

- 基础：intro2（1）；variables1 ~ 6（6）；functions1 ~ 5（5）；if1 ~ 3（3）；quiz1（1）；primitive_types1 ~ 6（6）。
- 所有权与容器：vecs1 ~ 2（2）；move_semantics1 ~ 6（6）；strings1 ~ 4（4）；hashmaps1 ~ 3（3）。
- 数据建模与组织：structs1 ~ 3（3）；enums1 ~ 3（3）；modules1 ~ 3（3）；quiz2（1）。
- 结果、抽象与生命周期：options1 ~ 3（3）；errors1 ~ 6（6）；generics1 ~ 2（2）；traits1 ~ 5（5）；quiz3（1）；lifetimes1 ~ 3（3）。
- 测试和迭代器：tests1 ~ 4（4）；iterators1 ~ 5（5）。
- 智能指针与并发：box1、rc1、arc1、cow1（4）；threads1 ~ 3（3）。
- 宏与代码质量：macros1 ~ 4（4）；clippy1 ~ 3（3）。

15. 继续更新这份笔记

完成新练习后，先核对其编译、测试或 Clippy 结果，再在对应主题增加“语法、返回类型、所有权影响、常见错误”四项；最后更新第 14 节名单和开头的完成数。现阶段未完成的 conversions、tests5 ~ 9、algorithm1 ~ 10 应在做完后再纳入。

参考：The Rust Programming Language、Rust 标准库、Rust Clippy。